Formats

Formats

This part contains 16 notes, from `formats/arrays-and-signatures.md` through `formats/tiletypes.txt`.

Notes under `formats/`, in path order, followed by the troff format notes `formats/glevnw01.txt`, `formats/gani.txt`, `formats/grmap001.txt`, and `formats/tiletypes.txt` converted to markdown.

`formats/arrays-and-signatures.md`

arrays.dat, arrays2.dat and the .sig files

Date: 2026-10-21

From client 2.05 rev 2 (traced), clients 2.17, 2.22 and 2.31 (disassembly of the load and check path), and the files in a 2.22 install. Tags: **2.05** · **[2.17]** · **[2.22]** · **[2.31]** · **[test]** · **[inf]** inferred.

Summary

File	Holds	Signed	Checked by
`arrays.dat`	25 built-in tile and sprite tables	no	nobody. The client quits with "File not found" if it is missing. **2.05**
`arrays2.dat`	Listserver hosts, port and URLs, one per line	`arrays2.sig`	2.22 (traced), 2.17 and 2.31 (same code)
`graal.sig`	Signature over the client exe itself		2.22 (traced), 2.17 and 2.31 (same code)

2.05 has no signature check at all. It doesn't reference `arrays2.dat`, `graal.sig` or `arrays2.sig`. **2.05**

arrays.dat

Big-endian 32-bit integers. Each array starts with a header word: **2.05**

```
header = (kind << 16) | count
kind 0: count int32 values follow
kind 1: count nested arrays follow (each with its own header)
```

The client reads exactly 25 arrays in a fixed order into 25 globals at startup. The 2.22 file (11,500 bytes) parses to exactly 25 arrays with nothing left over. **[test]**

#	Shape	Content (where usage was traced)
0	934 ints, max 4089	Tile ids. With array 1, the legacy walkability test: in the old tile mode, a tile counts as blocking unless it is in array 0 or array 1. **2.05**
1	116 ints	Tile ids, second list for that test
2	133 ints	Tile ids (use not traced)
3	76 ints	Tile ids (use not traced)
4	40 ints	Tile ids (use not traced)
5, 6	4 ints each	Single 2×2 tile blocks (708–711, 724–727) (use not traced)
7	79 ints	Tile ids. The legacy tile-type lookup maps a tile's position in this list to 4 classes (positions 0–11, 12–19, 20–62, 63+). The newer tile-type path returns types 6–9 for the same 4 classes. **2.05**

#	Shape	Content (where usage was traced)
8	4 lists, one per facing	Ledge tiles (tile type 21): a ledge jump needs the tile to be in the list for the player's facing. See `client-internals/ledges-push-pull.md` **2.05**
9–12, 14, 15	small nested lists of 2×2 tile groups	Used by level-object code (9 to 12: cut and lift, see Seihai's `cut-and-lift.md`)
16	2 × 2×2 blocks	Stone blocks: entry 0 pushable, entry 1 pullable. See `client-internals/ledges-push-pull.md` **2.05**
13	4 ints	One 2×2 tile block, used with array 14
17	124 × 4	Packed records used by the horse object **2.05**
18	nested	Use not traced
19	nested	Used by the leap object **2.05**
20, 22, 23	nested	Used by the baddy AI object **2.05**
21	4 × 4	Packed values (use not traced in detail)
24	356 ints, packed	Used by the item, bomb and extras objects **2.05**

All tile ids index the 4096-tile `pics1` sheet. Mapping every table to a gameplay rule needs more tracing, and isn't needed to load the file.

arrays2.dat

Plain text, CRLF lines. The 2.22 client needs at least 5 lines. Lines are numbered from 0. **[2.22]**

Line	Use
0	Main listserver host
1	Listserver port
2	Web host, joined into a URL template **[inf]**
3	Registration page URL
4	Translation service URL
5, 6	Fallback listserver hosts (optional)

The Graal Reborn copy points at `<list-host-reborn>` port 14922, with `<list-host-graalcc>` and `<list-host-graalbiz>` as fallbacks. **[test]**

The signatures

Both `.sig` files are 128 bytes, the size of an RSA-1024 signature. **[test]** The check runs at startup: **[2.22]**

1. The client builds a 1024-bit RSA public key from an obfuscated hex string assembled one character at a time in code. Each decoded byte is `not(b) - 10`.
2. It verifies `graal.sig` against **its own exe file**, then `arrays2.sig` against `arrays2.dat`. The verifier is an RSA crypto component configured with a SHA-1 signing digest and a PKCS#1 encryption scheme. **[inf]** That's from the component's class and property names. No signature has been verified yet, so the exact algorithm is not confirmed.
3. On any failure it shows "The Graal execution file is corrupted. Please check for viruses or other programs that manipulate…" and exits.

2.17, 2.22 and 2.31 embed the same two obfuscated key strings and the same error text. **[2.17] [2.22] [2.31]**

Open: checking the shipped signatures by hand against the decoded key with exponents 3, 17 and 65537 did not reproduce PKCS#1 padding, so the key encoding (exponent, byte order, or which of the two strings is used) is not pinned down yet.

The Graal Reborn 2.22 exe has the check disabled

The Reborn `Graal_222.exe` was rebuilt on the same day its `arrays2.dat` was edited. In its RSA verify routine, the three conditional branches that raise the verify error are rewritten as unconditional jumps, so verification always passes. The 2.17 and 2.31 exes have the original conditional branches. **[2.22] [2.17] [2.31]**

That's why the Reborn client runs with a modified `arrays2.dat` and an old `arrays2.sig`, and with a modified exe and the old `graal.sig`. An unpatched 2.17, 2.22 or 2.31 client would refuse both. **[inf]**

Practical upshot: pointing an unpatched 2.17+ client at another listserver means patching the exe, as Reborn did. Editing `arrays2.dat` alone fails the signature. 2.05 has no such file. Its listserver host is the DES-hidden string in `client-internals/saved-passwords.md`.

`formats/gmap.md`

GMAP (GRMAP001) as read by servers and clients

Date: 2026-10-21

Sources: GServer-v2 `TMap.cpp` (GPLv3, summarized); 2.05 string table; `grmap001.txt` (writer side). Tags as elsewhere.

`grmap001.txt` covers the three **writers** (dungeon generator, terrain generator, GraalLevelGen). This note covers the **reader** keys.

Keys read by GServer [srv]

Key	Args	Meaning
`GRMAP001`	—	signature (first line)
`WIDTH` / `HEIGHT`	n	grid size in levels
`GENERATED`	1 arg	read and stored. Not used to rebuild names on the server side
`LEVELNAMES` … `LEVELNAMESEND`	one row per line, quoted, comma-separated names	grid fill, row-major. Names are lowercased for lookup
`MAPIMG`	file	overview image
`MINIMAPIMG`	file	minimap image
`NOAUTOMAPPING`	—	flag
`LOADFULLMAP`	—	flag: preload every level
`LOADATSTART` … `LOADATSTARTEND`	name list	levels to preload

Each line is tokenized on whitespace after stripping `\r`. Lines inside the `LEVELNAMES` block are gs-untokenized (quoted CSV). **[srv]**

On the wire

A player on a gmap has `PLPROP_CURLEVEL` = the gmap name and `PLPROP_GMAPLEVELX/Y` = the grid cell. `PLO_PLAYERWARP2` (49) carries x, y, z, then the gmap cell x and y. `PLI_ADJACENTLEVEL` (35) preloads neighbors. **[srv]** A real 2.22 client crosses an edge with normal props only: it keeps the global position in the flags `gr.x` / `gr.y` (local + 64 * cell) and sends the wrapped local X or Y together with the changed cell prop (43 or 44), plus a fresh `PLI_TAMPERCHECK` (39). Details in `client-internals/feature-specs/06-gmap-overworld.md`. **[cap]**

2.05

2.05 rev 2 **does** contain the `GRMAP001` string and a reader for it that knows `WIDTH`, `HEIGHT`, `GENERATED`, `GENSEED`, `GENBASE`, `GENEVENBORDERS`, `GENHEIGHT`, `GENCHAOS`, `LEVHEIGHT`, `LEVCHAOS`, `MAPIMG`, `HEIGHTMAP...HEIGHTMAPEND`, `RANDOMSEEDS...RANDOMSEEDSEND` and `HEIGHTS`, plus a `TGraalMap` class. No static caller of that reader was found, so it may be dormant. 2.05 has no `PLO_PLAYERWARP2` handler (49 is missing from its jump table) and no `gmaplevelx` prop name. It references `bigmap.txt` and `minimap.txt` and has the GS1 commands `setmap`/`setminimap`/`loadmap`, so the text big-map and minimap overlays are the active path in 2.05. **2.05** (An earlier version of this note said 2.05 had no `GRMAP001` string; that was wrong. `client-internals/build-catalogue.md` also lists GRMAP001 for 2.05.) The `bigmap.txt`/`minimap.txt` format is described below. GServer's `PLO_BIGMAP` (171) and `PLO_MINIMAP` (172, e.g. `zodiacminimap.txt,zodiacworldminimap3.png,10,10`) are the later server-pushed versions.

2.05 big map and minimap text files 2.05

Defaults: - Big map: `bigmap.txt` with image `map.png`. - Minimap: `minimap.txt` with image `map2.png`. - Each map also has a few built-in integers. For the big map they are 10, 16, 1688 and 1024; for the minimap they are 664 and 896. - In training (offline tutorial) mode the client switches to `tutorialmap.txt` and `tutorialmap.png`, with 3, 4, 192 and 486.

Format: plain text, one map row per line. - Each line is parsed as a comma-separated list (Delphi `CommaText`, so quoting is allowed). - Each cell is normalised and appended to one flat list. - The column count is the number of cells on the **first** line, and the row count is total cells ÷ columns. - A cell is a level file name; `x` marks an empty cell. - The shipped `bigmap.txt` is 20 columns × 21 rows of quoted level names. The loader builds a map object of that grid with the image file name attached.

Per-server override: a server-list entry with more than the usual 8 fields can replace these settings. - Field 8 is the big-map file and field 9 its image. Fields 10 and 11 are two integers. - Field 12 is the minimap file and field 13 its image. Fields 14 and 15 are two integers. - The integers become points in the player object. They are probably image offsets or scale, which fits the default values above, but their use is not traced. **[inf]**

The GS1 commands `setmap` and `setminimap` load the same kind of file from a script.

`formats/gobjset01.md`

GOBJSET01 — object stamps

Signature `GOBJSET01`. Five sets ship in the client tree (`objectsStandard`, `objectsHouse Inside`, `objectsCastle`, `objectsSnow`, `objectsVolcano`).

```
OBJECT width height name
<height lines of width*2 base64 characters>
OBJECTEND
```

Same 12-bit base64 packing as GLEVNW01. The name may contain spaces. There is no layer, link, NPC, or type byte.

Placing a stamp writes the tile ids onto the board. Collision is decided later by the type table. Example: the first stamp in objectsStandard is 2×2 “Bush”, base64 `ACAD` / `ASAT` → ids 2, 3, 18, 19. Those type bytes are 20/22, so the bush is wall. The stamp file itself does not say so.

288 named multi-tile objects across the five sets. Stamp id 0 is a 4×4 ground fill in the dungeon generator; stamp id 1 is a hole and is not written.

`formats/graalsav.md`

Save games: Graalsav1 to Graalsv11, and the DES-wrapped `.code` / `.gsave` files

Date: 2026-10-21

Covers the clients from 1st Graal (Sept 1998) to 1.32 (Jan 2000). The notes come from static analysis of the shipped executables. No decompiler output is reproduced. Field names come from the 13th Graal script variables (`playerrupees`, `playerdarts` …), whose storage lines up with the save record of the same build. Earlier and later builds are matched by position. Names that rest only on position or defaults are marked *inferred*.

Container

Every version is a Delphi `TWriter` value stream, not a hand-made binary layout. Each value starts with a one-byte `TValueType` tag:

Tag	Type	Body
2	Int8	1 byte
3	Int16	2 bytes LE
4	Int32	4 bytes LE
5	Extended	10-byte x87 float
6	String	1-byte length and chars
12	LString	4-byte length and chars (strings of 256 bytes or more)
8 / 9	False / True	none

`WriteInteger` picks the smallest of Int8, Int16 and Int32, so a reader has to go by the tag. The first value is always the magic string (`Graalsav1` … `Graalsav9`, `Graalsv10`, `Graalsv11`).

Versions by build

Magic	Written by	Read by	Extension	Wrapped
Graalsav1	1st, 2nd	same	`.save`	no
Graalsav2	4th alpha, 5th, 5th alpha	same	`.save`	no
Graalsav3	6th	same	`.save`	no
Graalsav4	7th, 8th	same	`.save`	no
Graalsav5	9th alpha to 12th	same	`.save`	no
Graalsav6	13th	same	`.save`	no
Graalsav7	14th, 15th	same	`.save` (14th), `.gsave` (15th)	15th on
Graalsav8	1.1, 1.1.9	7 and 8	`.gsave`	yes
Graalsv11	1.21 to 1.41 rev1 (writer traced in 1.21 and 1.32; later builds by string references)	9, 10 and 11	`.gsave`	yes

3rd and 4th Graal reuse Graalsav1 (the 4th alpha was built later than 4th, as the build catalogue notes). Earlier catalogue rows listed 1.21 to 1.39 as "Graalsav9". Those builds read 9 too, but they write `Graalsv11` and also

read `Graalsv10`, and a search for `Graalsav` misses both. The record layout was read from 1.21 and 1.32. In 1.33 to 1.41 rev1 the three magics are referenced the same way as in 1.32 (two references to `Graalsv11`, one each to the readers' `Graalsav9` and `Graalsv10`), so they write `Graalsv11` too, but the record itself was not re-read. No build in the set writes `Graalsav9` or `Graalsv10`. The v9 and v10 layouts below come only from the 1.21 to 1.32 readers, so they presumably match saves from a 1.2x build that isn't in the archive. From 15th Graal saving only works on a level from the Cyberjoueur server or on the online start level. From 1.21 the client refuses to save in online mode: "The server saves your game if you have an own account."

File layout

```
magic                    String
mode                     Integer   0 = one player, 1 = two players, 2 = online, 3/4 = three/four
players (14th on)
[mode 2]      String, String         two online-session strings (*inferred*: nickname and server)
player record × n        n = 1, 2, or the player count in 3/4-player modes
[Graalsav2+]  flag list    Integer count, then count × String
```

From `Graalsv11` the header is just `magic`, then `Integer players-1`, then the records and the flag list.

Player record

Graalsav1

```
String  level
String  nickname
Float   x
Float   y
Integer dir                  (*inferred*)
Integer fullhearts           (*inferred*)
Float   hearts
Integer rupees
Integer darts                (default 5)
Integer bombs                (default 10)
Integer glovepower           (*inferred*, default 2)
4 × Boolean                  (flags; the first defaults to true)
4 × Boolean
4 × Integer colours          (the same four values the relay `S` message sends)
```

Graalsav2 to Graalsav6

The same start (level, nickname, x, y, dir, fullhearts, hearts, rupees, darts, bombs), and then:

Field	2	3	4	5	6
glovepower	✓	✓	✓	✓	✓
shieldpower, swordpower	✓	✓	✓	✓	✓
shootpower			✓	✓	✓
one Boolean	✓	✓			
5 × Integer colours	✓	✓	✓	✓	✓
Integer headset	✓	✓	✓	✓	✓
horse: String image, Integer bushes				✓	✓

Field	2	3	4	5	6
carried NPCs: Integer n, then n × (String image, String script)				✓	✓
trailing Boolean					✓

The power order in the file is glove, shield, sword, then shoot. The colour count goes from 4 to 5 between Graalsav1 and Graalsav2.

Graalsav7 and Graalsav8

The same record as Graalsav6. Graalsav8 adds one trailing Integer, and the 1.1.9 loader reads it only when the magic is 8 or above. The change from 6 to 7 is in the file header (the four-player mode).

Graalsav9 to Graalsv11 (1.21 to 1.32)

`Graalsv11` is taken from the writer. The `[v9]` / `[v11]` conditions come from the reader.

The player is no longer a list of typed fields. It is one string in the network player-props encoding:

```
String  props          concatenation over prop index i: chr(i+32) + the prop's encoded value
[v9]    String horse image, Integer bushes
Integer n, then n × (String image, String script)        carried / inventory NPCs
[v11]   Integer w, then w × String                        NPC weapon names
[v11]   Integer selected weapon index
```

So the save uses the same props codec as the GServer protocol (`protocol/gserv-1999.md`). A props decoder written for gen-2 packets can read these saves.

DES-wrapped files

From 5th Graal the client carries a small DES class and two 8-byte keys, `AXBORG15` and `VERGRAAL`. They protect local files against editing. Nothing on the network is encrypted in these builds.

`.code` files (5th Graal on)

These are `guildname.code` (behind the `playeringuild` script check) and, in 15th Graal to 1.1.9, `reg.code` (the registration key).

```
24 bytes  DES-ECB(AXBORG15, P)
24 bytes  DES-ECB(VERGRAAL, P)
```

`P` is a 24-byte buffer that holds a Pascal short string (length byte, then up to 23 characters). The reader decrypts both halves, requires them to be equal, and returns the string. Anything else (a file too short or halves that differ) reads as `###`. In 15th Graal to 1.1.9 the registration key is also broadcast as relay type `8`, so two players with the same key are noticed.

`.gsave` (15th Graal on)

```
u32 LE    n = length of the padded plaintext
n bytes   DES-ECB(AXBORG15, plaintext)
n bytes   DES-ECB(VERGRAAL, plaintext)
```

The plaintext is the `TWriter` stream described above, padded to a multiple of 8 with random bytes. The loader decrypts both copies, compares them byte for byte, and parses the result only if they match.

The DES class takes a second constructor argument (7) whose role hasn't been pinned down. `client-internals/saved-passwords.md` (PR #6, not yet merged; summarised in `client-internals/des-and-hidden-strings.md`) found that the 2.05 DES is textbook DES-ECB with each key byte bit-reversed. The same class name and key handling are probably at work here, so a round-trip test should try `AXBORG15` and `VERGRAAL` bit-reversed first. Before anyone writes a tool for these files, one real save should be round-tripped.

`formats/grv1-decoder/README.md`

grv1.py: GR-V1 / Z3-V1 level decoder

Decodes binary `.graal` (`GR-V1.00`–`GR-V1.04`) and `.zelda` (`Z3-V1.02`–`Z3-V1.04`) levels. Python 3, no dependencies. Original code written from `formats/level-binary.md` and `client-internals/level-editor-205.md`.

```
python3 grv1.py level.graal           # summary: links, baddies, NPCs, chests, signs
python3 grv1.py level.graal --board   # plus the 64x64 tile grid
python3 grv1.py level.graal --json    # JSON (add --board for tiles)
```

Behaviour follows the 2.05 client loader:

- Code width is 11 bits for `Z3-V1.02`, 13 bits for `GR-V1.02`, `GR-V1.03` and `GR-V1.04`, and 12 bits otherwise.
- NPCs are read from `GR-V1.00` on, chests from `GR-V1.01` on, including `GR-V1.04`. `.zelda` files have neither.
- After the signs, an empty line and a `HEIGHTS` … `HEIGHTSEND` block are kept (at most 9 rows of 9 decimals). `GR-V1.04` is that block on a `GR-V1.03` body. No real `Z3-V1.02` file is in the notes; the 11-bit width is the Java rule in `client-internals/precursors-and-fan-builds.md`.
- The baddy section ends at the first record shorter than 3 bytes or whose x byte is ≥ 0x80. An x byte of 0 is a valid baddy (real levels have them).
- NPC scripts have `0xA7` turned into newlines. Sign text goes through the sign alphabet, and `#K(13)` (CR) is dropped.

Checked against the 13 tutorial levels embedded in the 2.05 client and the 9 in 2.22. All parse with the expected link grid, chest items, and sign text (see `client-internals/offline-tutorial-205.md`).

To get the tutorial levels, extract the `TUTORIAL*` RCDATA resources from your own copy of the client (any PE resource tool works). The levels are not committed here.

`formats/gwebl001-cache.md`

GWEBL001: the web-level cache (`weblevels/*.code`)

Date: 2026-10-21

Traced from the 2.05 rev 2 cache reader and writer, then confirmed on 144 real cache files from a 2.22 install: every one decrypts, passes the checksum, parses, and re-encrypts byte for byte. Tags: **2.05** read from the client · **[2.31]** read from the 2.31 client · **[test]** confirmed on real files.

What it is

A local copy of a server level, keyed by the level's modification time, so the client can skip downloading a level it already has.

- **Written** when the server sends the level's 5-byte modification time (server packet id 39, `PLO_LEVELMODTIME` in GServer naming). The client saves the level it currently holds, with that time. **2.05**
- **Read** when the client enters a level: if a cache file exists, the client builds the level from it. **2.05** It presumably then only needs the server to send levels that have changed. **[inf]**
- A file whose header doesn't match the current server is **deleted**. **2.05**

Path

```
<exe dir>\weblevels\<server host>\<level>-<server port>.code
```

`<level>` is the level's file name, lowercased, with any path removed. The reader also tries `<level>.code` without the port. **2.05** Real example: `weblevels\<game-server-host>\bank.nw-14803.code`. **[test]**

File layout (encrypted)

```
int32 LE   N            = padded plaintext length + 8
N-8 bytes  ciphertext   DES-ECB, one fresh key per 8-byte block
8 bytes    checksum     bytewise sum (mod 256) of all plaintext blocks, per byte position
```

The plaintext is padded with `0x0A` to a multiple of 8. **2.05**

Per-block keys

The key for each block is 8 calls to Delphi `Random(256)`, drawn fresh for every block in file order. The generator is seeded first:

```
RandSeed = 0x04A80B38
for each byte c of <level> (lowercased name, no path):
    RandSeed = RandSeed * c            (32-bit wrap)
```

Delphi `Random(n)`: `RandSeed = RandSeed * 0x08088405 + 1`, then `(RandSeed * n) >> 32` (unsigned). The DES key bytes are bit-reversed as in `client-internals/saved-passwords.md`. **2.05 [test]**

The "encryption" therefore needs only the level name. A wrong name shows up as a checksum mismatch.

Plaintext layout

```
"GWEBL001"
GCHAR len + server host          must equal the connected host
GCHAR hi, GCHAR lo               server signature byte, as (hi-32)*128 + (lo-32)
5 bytes                          level modification time, 5 × 7-bit GCHAR, big-endian
GCHAR len + level name           must equal <level> above
level body                       GR-V1.03 (2.05) or GR-V1.03 / GR-V1.04 (2.31)
```

The signature check passes when the stored value equals the value the server sent in packet id 25 (`PLO_SIGNATURE`), or when the stored value is 1 and the server sent 73 (GServer's usual value). **2.05** All 144 test files hold 73. **[test]**

Decoding the modification time as a 35-bit integer gives Unix time. The test files decode to dates a little before their file timestamps (for example `bank.nw` decodes to 1247727798, 16 July 2009, and the file is dated 26 July 2009). **[test]**

Level body as cached

The body is an ordinary GR-V level (`level-binary.md`, `level-binary-client.md`), but the writer fills only some sections:

Section	Cached
Board	yes, 13-bit run-length stream
Links	yes, `dest x y w h newx newy` lines, then `#`
Baddies	no, terminator only (`FF FF FF 0A`)
NPCs	no, empty (#)
Chests	no, empty (#)
Signs	yes, `x+32, y+32, sign text, \n`
Heights	2.31 only, as a `HEIGHTS` block when the level has a height map, which also switches the magic to `GR-V1.04` **[2.31]**

Tool

`tools/file-formats/gwebl.py`:

```
python3 gwebl.py weblevels/<game-server-host>/bank.nw-14803.code       # summary
python3 gwebl.py FILE.code --json                                      # full parse
python3 gwebl.py FILE.code --raw plain.bin                             # decrypted stream
```

The module also has `encrypt_stream` (re-creates files byte for byte), `read_board` / `write_board`, and `parse_grv` for plain `.graal` data.

`formats/level-binary-client.md`

GR-V levels as the client reads them (2.05, 2.31) and GR-V1.04

Date: 2026-10-21

Companion to `level-binary.md`, which is built from server source. This file is from the clients themselves: the 2.05 rev 2 GR-V reader and writer, and the 2.31 reader and writer. The board rules were confirmed by decoding

144 real cached levels and 13 embedded tutorial levels and re-encoding every board byte for byte. Tags: **2.05** · **[2.31]** · **[test]** · **[inf]** inferred.

Agreement with `level-binary.md`

1. **Code width.** 13 bits for `GR-V1.02` and `GR-V1.03` (and `GR-V1.04` in 2.31), and 12 bits for `GR-V1.00`, `GR-V1.01` and both `Z3-V1.*`. `level-binary.md` now says the same. **2.05 [2.31]**
2. **Double runs.** The 2.05 writer emits them, as `level-binary.md` now records. **2.05 [test]**
3. **Sign terminator.** 2.05 reads signs to end of file. 2.31 stops at the first empty line, because a `HEIGHTS` block can follow. **2.05 [2.31]**

Board writer (2.05)

Walks the 4096 tiles in order and picks, at each position:

Case	Codes emitted
Next tile equals this one	`0x1000 | n`, then the tile. `n` counts up to 64 equal tiles.
Tiles `i, i+1` equal `i+2, i+3` (pattern AB AB)	`0x1100 | n`, then A, then B. `n` counts up to 64 pairs.
Otherwise	the tile (`& 0xFFF`)

Codes are packed 13 bits each, least significant bit first, and the last partial byte is flushed. **2.05** `write_board` in `tools/file-formats/gwebl.py` follows these rules and reproduces every board in the test set exactly. **[test]**

Section rules in the client reader (2.05)

Section	Read for
Links, #-terminated	all versions
Baddies, records until one whose first byte is ≥ 0x80 (the `FF FF FF` terminator)	all versions
NPCs, #-terminated	GR-V1.00 and later
Chests, #-terminated	GR-V1.01 and later
Signs	everything left

A baddy record is read up to a `\n` that comes after at least 3 bytes, so the 3 binary bytes may contain `0x0A`. **2.05**

Sign text in files

As `sign-text.md` says, signs in `.graal` files use **the same alphabet as signs on the wire**: `byte - 32` indexes the table

```
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!?-.,#>()#####"####':/~&### <####;\n
```

and # entries are escape codes (`#A #B #X #Y #u #d #l #r #h #x #y #z #4 #.`). **2.05** The token `v*eUWf` + `0x80` decodes to `#K(13)` followed by a newline. It is the explicit line-break marker, not a space. **[test]** `sign_text` in `gwebl.py` implements this.

Signs at (0,0) are common in the test levels. They hold dialogue text, so they are probably messages that NPC scripts show by index rather than signs placed on the map. **[inf]**

GR-V1.04 (2.31)

GR-V1.04 is **GR-V1.03 plus an optional height map**. **[2.31]**

- The writer emits `GR-V1.04` only when the level has a height map, and `GR-V1.03` otherwise.
- After the signs it writes an empty line, then:

```
HEIGHTS
h00,h01,…,h08
…                        9 lines of 9 comma-separated decimals
h80,h81,…,h88
HEIGHTSEND
```

- The reader accepts this block after the signs whatever the magic says. `HEIGHTSEND` is matched case-insensitively, and at most 9 rows of 9 values are kept.
- The 9×9 grid is the same `HEIGHTS` overlay that `GLEVNW01` carries (`glevnw01.txt`) and that TerrainGenerator writes (`tools/generators.md`). A 9×9 grid fits the corners of an 8×8 grid of 8×8-tile cells on a 64×64 level. **[inf]**
- Board coding is unchanged: 13 bits, same run codes.

In 2.31, GR-V parsing appears only in the web-level cache code. That's the one place the client both writes and reads GR-V1.04. **[2.31]**

Embedded tutorial levels (2.05)

The 2.05 exe carries the offline tutorial as 13 `RCDATA` resources, all plain GR-V1.03: **2.05**

Resource	Level name
`TUTORIALA1…A4`	`tutorial_a-1.graal` … `tutorial_a-4.graal`
`TUTORIALB1`, `B2`, `B3`, `B4`	`tutorial_b-1.graal` … `tutorial_b-4.graal`
`TUTORIALB2H`	`tutorial_b-2_house.graal`
`TUTORIALC1…C4`	`tutorial_c-1.graal` … `tutorial_c-4.graal`

When the client is **not connected** and the requested level's name starts with `tutorial`, it opens the matching resource instead of a file on disk. Otherwise levels come from disk. **2.05** The levels form a 3×4 grid joined by edge links. The house in B2, the B4 level and the B2 level hold all the NPCs, chests and signs. **[test]**

`tools/file-formats/extract_tutorials.py Graal.exe OUTDIR --json` writes the 13 `.graal` files and a parsed JSON for each.

`formats/level-binary.md`

Binary level formats: GR-V1 (.graal) and Z3-V1 (.zelda)

Date: 2026-10-21

Source: GServer-v2 `TLevel::loadGraal`, `TLevel::loadZelda` (GPLv3, summarized). Tags: **[srv]** server source · **2.05** · **[inf]** inferred.

The 2.05 string table holds exactly these level magics: `Z3-V1.03`, `Z3-V1.04`, `GR-V1.00`–`GR-V1.03`, and `GLEVNW01`, each set appearing three times in the string table (one copy sits next to the GWEBL001 web-level

cache code). **2.05** The 2.05 loader itself has not been traced, so the bit-stream rules below come from server source.

Correction

CONSOLIDATED.md calls GR-V1 a "64×64 board of 2-byte tiles". It is not. The board is a **packed bit stream with run-length codes**. Only `.nw` text and the `PLO_BOARDPACKET` wire form store one 16-bit value per tile.

Layout

```
char[8] magic        GR-V1.00 | GR-V1.01 | GR-V1.02 | GR-V1.03 | Z3-V1.03 | Z3-V1.04
bitstream board      until 4096 tiles are produced
text   links         lines until "#" or an empty line
binary baddies       3-byte records until FF FF FF, each followed by a verse line
text   npcs          lines until "#" or an empty line  (not in .zelda)
text   chests        lines until "#"   (GR-V1.01+ only, not in .zelda)
text   signs         lines until an empty line or EOF
```

Board bit stream

- Code width: **12 bits** for `GR-V1.00`, `GR-V1.01`, and `Z3-V1.03`/`Z3-V1.04`; **11 bits** for `Z3-V1.02`; **13 bits** for `GR-V1.02` and `GR-V1.03` only. The 2.05 loader compares the magic against 1.02 and 1.03 to pick 13 bits. **2.05**
- Codes are read least-significant-bit first: bytes are appended to a little-endian bit buffer and `width` bits are taken from the bottom.
- Flag bit = the top bit of the code (`0x800` at 12 bits, `0x1000` at 13 bits).
  - **Flag set:** a control code. `count = code & 0xFF`. If `code & 0x100` is set, the next run is a **double** run.
  - **Flag clear:** a tile id.
- When `count == 1` (the default), write the tile once.
- Single run: write the next tile `count` times, then reset `count = 1`.
- Double run: read two tiles A and B and write `A B` `count` times (`{double 4} 5 6` → `5 6 5 6 5 6 5 6`), then reset.
- Tiles fill `board[x + 64*y]` row by row, layer 0 only.

12-bit files cannot address tiles ≥ 2048 (the flag bit takes the top bit). That is why 1.02 widened the code to 13 bits for the 4096-tile `pics1`. **[inf]**

Records

Section	Shape	Notes
Link	`dest x y w h newx newy`	Spaces in the level name are allowed. Tokens before the last 6 are rejoined. Links to missing levels are dropped. `newx`/`newy` may also be `playerx`/`playery`, optionally followed by a signed number (`playery+10`); see `client-internals/decomp-findings-2026-10-25-f/02-link-arrival-expressions.md`.

Section	Shape	Notes
Baddy	`i8 x, i8 y, i8 type` + `\n`-terminated verse line	Verses separated by `\`: sight, hurt, attack. Z3-V1.03 has no verse line. Terminator is `FF FF FF` followed by an empty line.
NPC	`GCHAR x, GCHAR y, image#script`	The script uses `0xA7` for newline. An empty image means a script-only NPC.
Chest	`GCHAR x, GCHAR y, GCHAR item, GCHAR signindex`	The item index is in `npc-library/gs1-reference-205.md`.
Sign	`GCHAR x, GCHAR y` + sign-alphabet text	Decoder in `sign-text.md`.

Coordinates in these records are whole tiles (GCHAR), not half-tiles. **[srv]**

Z3-V1 differences

- Sections: board, links, baddies, signs. **No NPC and no chest section.**
- Baddy verse line only in `Z3-V1.04`.
- Board coding is the same as GR-V1.00 (12-bit).
- `Z3-V1.02` also exists. It uses **11-bit** codes and only the 1998 Java Zelda Online games read it. In `Z3-V1.02` and `Z3-V1.03` the Java code reads each baddy as a 4-byte record whose last byte is always `\n`. See `client-internals/precursors-and-fan-builds.md`.
- Checked against 139 distinct real files; details in `client-internals/decomp-findings-2026-10-25-j/`:
  - `Z3-V1.03` links are written with decimals (`31.0 42.0 2.0 1.0 43.5 41.0`), so numbers are floating point. A negative arrival coordinate means "keep the player's own coordinate".
  - A computer record is three raw bytes `x y type` and then the rest of the line. The rest is empty in `Z3-V1.03` (4-byte records) and the verse text in `Z3-V1.04`. `0x0A` can be an `x` or `y`, so take the three bytes before looking for the line end.
  - The reader must decide by the magic, not the extension: `.zelda` names hold `GR-V1` files and `.graal` names hold `Z3-V1.04` files.
  - Types seen are 0 to 6 (Zelda Online's names: green, blue and red soldier, shot soldier, swamp soldier, jumping hare, octopus).

Open

- ~~GR-V1.04 changes unknown.~~ Answered in `level-binary-client.md`: GR-V1.04 is GR-V1.03 plus an optional 9×9 `HEIGHTS` block after the signs (2.31). **[2.31]**
- ~~Whether the 2.05 writer emits double runs.~~ Answered: it does. The writer emits a single run for 2 to 64 equal tiles and a double run (`ABAB…`, up to 64 pairs) where the pattern starts; it always writes `GR-V1.03` for non-`.nw` files. Single runs of exactly 64 occur in the embedded tutorial levels. **2.05**

`formats/seihai-boards.md`

Boards and files

Two names, one board

A board is accepted when the file name is in a known pair and the header is a known magic. `acceptLevelBoard` requires both. The pairs are aliases, not different layouts.

```
flowchart LR
  subgraph names [File name]
    seihai[".seihai"]
    graal[".graal"]
    as[".as"]
    nw[".nw"]
  end
  subgraph magics [Header]
    sh["SH-V1.00 … SH-V1.03"]
    gr["GR-V1.00 … GR-V1.03"]
    glevas[GLEVAS01]
    glevnw[GLEVNW01]
    z3["Z3-V1.03 · Z3-V1.04"]
  end
  seihai --- graal
  as --- nw
  sh --- gr
  glevas --- glevnw
  seihai --> gate[acceptLevelBoard]
  graal --> gate
  as --> gate
  nw --> gate
  sh --> gate
  gr --> gate
  glevas --> gate
  glevnw --> gate
  z3 --> gate
```

Default, written by Seihai	Still opens
`.seihai` with `SH-V1.00` through `SH-V1.03`	`.graal` with `GR-V1.00` through `GR-V1.03`
`.as` with `GLEVAS01`	`.nw` with `GLEVNW01`

`Z3-V1.03` and `Z3-V1.04` stay on the list as older headers. Comparison of the extension is case-insensitive. A `.txt` name, or a magic that is not in `LEVEL_MAGICS`, does not open.

Shipped boards use the new spellings. Overworld files are `.seihai` / `SH-V1.03`. The house and the cave, which used to carry the NW line, are `.as` / `GLEVAS01`.

The grid

`BOARD` is 64. `TILE` is 16. `WORLD` is 1024 pixels on a side. Cell `(x, y)` is index `y * 64 + x` in a `Uint8Array`.

Id	Name	Solid	Notes
0	Grass	no	Floor outdoors
1	Path	no	
2	Water	yes	
3	Deep	yes	
4	Tree	yes	Cleared if a landmark is stamped on it
5	Bush	yes	Sword or bomb removes it. One in five leaves a green rupee
6	Wall	yes	
7	Cave floor	no	Floor inside the cave
8	Cave wall	yes	

Id	Name	Solid	Notes
9	House wall	yes	
10	Tall grass	no	Slows movement to 0.62
11	Sand	no	
12	Bridge	no	The only walk on the meadow river
13	Wood	no	Floor inside the house
14	Crack	yes	Bomb only. Recorded in the save
15	Stair	no	Used as a link tile, not as a special tile type
16	Flower	no	
17	Fence	yes	
18	Heal	no	E while standing on it or facing it fills hearts

Solid means the body cannot enter the cell. Water, deep water, cracks, and bushes are solid. Stairs, paths, bridges, and heal tiles are not.

The five boards

```
flowchart LR
  ona["onA.seihai\nTutorial Glade\nSH-V1.03"]
  meadow["meadow.seihai\nNorth Meadow\nSH-V1.03"]
  house["house.as\nRowan's House\nGLEVAS01"]
  cave["cave.as\nHollow Cave\nGLEVAS01"]
  grove["grove.seihai\nThorn Grove\nSH-V1.03"]
  ona -->|north gate| meadow
  meadow -->|south gate| ona
  ona -->|house door| house
  house -->|door| ona
  meadow -->|cave mouth| cave
  cave -->|stairs| meadow
  meadow -->|east edge| grove
  grove -->|west edge| meadow
```

Id	File	Header	Who is there
`ona`	`onA.seihai`	`SH-V1.03`	Rowan. A slime, a bomb chest behind a block, a north warp and a door
`meadow`	`meadow.seihai`	`SH-V1.03`	Mara's shop. Slimes and octoroks. Bridge, cave mouth, east gate. A chest with sword III
`house`	`house.as`	`GLEVAS01`	Quill. Bed (`HEAL`). A closet block that pulls. Chest with sword IV
`cave`	`cave.as`	`GLEVAS01`	Ember. Two bomb cracks. Bats, a slime. Sword II, then a far chest
`grove`	`grove.seihai`	`SH-V1.03`	Wick. Thornback

A link is an axis-aligned rectangle in tile coordinates plus a destination id and tile. Standing on it after the gate timer calls `enter`, which logs `warpto`. Links are not a tile id. The gold tint on a link rect is only a draw.

How a board is built

Each `build*` function fills a `Uint8Array`, stamps rectangles, disks, and paths, then returns a `LevelDef`: id, file name, display name, header, biome (`overworld`, `cave`, or `house`), tiles, spawn tile, links, NPCs, chests, opps, pickups, blocks, and signs.

Scatter uses a seeded mulberry32 so the meadow and the grove grow the same trees every load. `clearAround` cuts disks of floor where people, gates, and signs must stand. `enter` still runs `freeLandmarks` in case a stamp overlapped.

Biome chooses the replacement floor when a bush or crack is destroyed: cave floor, wood, or grass.

Header gate

```
flowchart TD
  start[enter level id] --> found{LEVELS has that id?}
  found -->|no| refuse[Say Unsupported file type]
  found -->|yes| ext{Extension is seihai, graal, as, or nw?}
  ext -->|no| refuse
  ext -->|yes| magic{Header is SH, GR, GLEVAS01, GLEVNW01, or Z3?}
  magic -->|no| refuse
  magic -->|yes| copy[Copy tiles and apply the save]
```

`isSeihaiName` is true for `.seihai` and `.graal`. `isAsName` is true for `.as` and `.nw`. Either name may carry either header family. That is the compatibility rule: the extension pair and the magic pair are each aliases, and a known name plus a known magic opens.

`formats/seihai-systems.md`

Systems

Numbers below are what the source does, not targets. Times are seconds. Distances are pixels unless a value is named as a tile.

Body and collision

The player is tested as four corners around the feet: `(-5, -12)`, `(+4, -12)`, `(-5, -2)`, `(+4, -2)`. A corner that leaves the 64×64 board, hits a solid tile, or shares a cell with a block, NPC, chest, or sign refuses the move.

X is attempted, then Y, so a slide along a wall still works. After a refused axis, if Shift is not held, the engine tries to push. Push requires contact for 0.28 s, then the block moves one tile if `tileOpen` says the destination is in-bounds, not solid, and not occupied.

Pull happens only when the move succeeded and Shift (or the gamepad grab) is held, along a single axis. The block must sit on the tile the player just left. It steps onto that tile. The log says `pushed` or `pulled`.

The player is clamped to the world: x in `[8, 1016]`, y in `[12, 1020]`.

Sword

```
flowchart TD
  press[Attack held and cooldown clear] --> swing[swordT = 0.22, cooldown = 0.34]
  swing --> window{swordT between 0.02 and 0.20?}
  window -->|yes| box[Hit box in facing direction]
  box --> opp[Damage equals swordpower, then a short shove]
  box --> bush[Bushes in the box become floor]
  window -->|no| wait[No hit this tick]
```

Reach is `8 + sword * 4` pixels. An opp can be hit once per swing (`cool` 0.25 s). Bushes in the box are cut. About one in five, by a hash of the tile, drops one green rupee with an id starting `cut-`. Those drops are not written into `save.pickups`.

Sword power is an integer 1–4. Chests raise it only when the chest's sword is higher. A lesser blade is described and does not replace the one you have. Labels are Sword I through Sword IV.

Bombs and arrows

A bomb spends one from the pouch and sits on the player's current tile with a 1.35 s fuse. The blast covers a 36 px radius. Cracks and bushes inside it become floor. A crack is appended to `save.broken` as `levelId:x,y`. Opps inside the radius plus 8 px take 4 damage. The player takes 2 if they are inside the radius, before the hurt multiplier.

An arrow spends one, sets a 0.28 s cooldown, and flies at 210 px/s for 0.7 s or until a solid tile. Friendly arrows deal 2. They do not destroy cracks. Octoroks fire the other way: 110 px/s, 1.1 s life, 1 damage, only with a clear sampled line and the player nearly on the same row.

"No bombs." is said when the pouch is empty. An empty quiver simply does not fire.

Opps

Kind	Max hp	Motion	Touch damage
Slime	3	Hop toward the player inside 170 px, else wander	1
Octorok	3	Pace on x and reverse at walls. Shoots on a shared row	1
Bat	2	Chase at 42 px/s plus a weave	1
Thornback	12	Same hop as a slime, faster, aggro inside 280 px, larger body	2

Thorn's `hurtMul` of 2 adds 1 to each of those maxima and doubles incoming damage to the player after rounding, with a floor of 1.

On death, a normal opp says "The opp drops." Thornback says "Thornback falls.", sets `dead:<id>` so the boss does not return, and if `heart:thornback` is not set, grants one heart container (+2 max hp, +2 hp capped). Every kill also rolls a drop: heart, 5 blue rupees, or 1 green, by a hash of the position. Drop ids start with `drop-` and are not saved as placed pickups.

Contact knockback is 8 px along the line from the opp to the player. The player is invulnerable for 0.85 s after any hit (`iframe`). A red flash lasts 0.35 s and the hero blinks.

Talk, chests, shop, bed

`interact` on a talk edge, nearest first:

1. An NPC within 28 px. Mara is the shop. Anyone else opens the reading panel on the next line in their list, cycling.
2. A closed chest within one tile. Grants its rupees, bombs, arrows, heart container, and sword, then marks the id in `save.chests`.
3. A sign within one tile. The panel title is "Sign".
4. A heal tile underfoot or on the faced tile. Hearts fill. No panel.

The reading panel freezes the sim. Close, E, Space, or Enter dismisses it. Enter outside a panel focuses chat instead.

Mara sells one heart container for 40 rupees, flag `shop-heart`. After that she has nothing left. If the pouch is short she repeats the price.

`playerenters` is not a key. It is the log line written when a level with an `enter` script is loaded. The line also becomes the status text for that arrival.

Chat, up to 120 characters:

Line	Effect
`/save`	Capture and write the save now
`/where`	File name, tile, and swordpower
anything else	Logged as the nickname. If an NPC is within 30 px, they answer with their next line in the log, not the modal

Death and pause

At 0 hp the tick persists and stops the sim. The death panel's Wake button, or a talk edge, calls `revive`: full hearts, 85% of rupees kept (floor), position reset to that level's spawn, a short iframe, "You wake at the gate."

Pause lists visited files (the save stores level ids; the HUD shows file names). Resume toggles pause. Server list captures, persists, stops the frame loop, and returns to the launcher.

Save document

Key: `graal-client-save-v1:<worldId>`. Nickname key, separate: `graal-client-nick`. The prefix is the original storage name so a save written before the Seihai rename still continues.

```
flowchart LR
  tick[Dirty gameplay] --> wait[Wait until 4 s since last write]
  wait --> cap[capture]
  manual["/save, quit, death, revive"] --> cap
  cap --> json[JSON SaveV1]
  json --> ls[localStorage]
  ls --> boot[Next Continue]
  boot --> enter[enter boot=true]
```

`SaveV1` fields: `v` (must be 1), `world`, `nick`, `level` id, pixel `x` and `y`, `dir`, `hp`, `maxHp`, `rupees`, `bombs`, `arrows`, `sword`, `flags`, `chests`, `pickups`, `broken`, `blocks` by level id, `visited`.

`capture` writes the live player and the current level's blocks into that object, and dedupes the string lists. `loadSave` rejects a missing key, a bad version, a world that does not match, or a level id that is not in `LEVELS`. Missing arrays are filled so an older document still loads.

Autosave runs from the tick when `saveDirty` has been set and four seconds have passed. Opening a chest, spending a bomb, moving a block, taking a hit that kills, and changing levels all mark the save dirty. A warp captures before the new level replaces the old one, so the level you left keeps its block positions.

Loot on the shipped boards

Where	What
Glade chest, behind the block	6 bombs, 8 rupees
Meadow chest	Sword III if higher, 12 arrows
Mara	Heart container, 40 rupees, once
House chest, behind the pull	Sword IV if higher, 25 rupees
House floor	20 red rupees
Cave, past the first crack	Sword II if higher, 4 bombs
Cave, past the second crack	40 rupees, 10 arrows, a heart container
Thornback	Heart container the first time, plus a normal drop

Placed pickups on the grass are greens, blues, hearts, and a small bomb pile in the cave. Collecting one stores its id. Cutting a bush or killing an opp does not.

`formats/sign-text.md`

Sign text decoding

Date: 2026-10-14

Binary sign text uses **one** packed alphabet. The same encoding is used in `.graal` / `.zelda` sign sections and on the wire (`PLO_LEVELSIGN`). GLEVNW01 text signs are plain text. (An earlier version of this note described a separate "file alphabet" with `char + 89`. That was wrong; see "Correction" below.)

Encoding

A sign record is `GCHAR x`, `GCHAR y`, then the text bytes. Each text byte is a GCHAR: `byte - 32` is an index into a fixed 97-entry alphabet. The alphabet is the same string in the 2.05 client and in GServer-v2 (`TLevelSign`).

- Codes 0–61: `A`–`Z`, `a`–`z`, `0`–`9`.
- Codes 62–96: punctuation, space (code 89, byte `0x79`) and newline (code 96, byte `0x80`). Several slots in this range are # placeholders.
- # slots are glyph codes. One or two codes stand for # plus a symbol: `#A` `#B` `#X` `#Y` (button icons), `#u` `#d` `#l` `#r` (arrows), `#x` `#y` `#z`, `#h` and `#4` (two codes each), and `#.`. Code 86 is a literal #.
- Characters outside the alphabet are written as the escape `#K(n)`, with `n` the decimal character code, spelled out in alphabet codes.
- Bytes under 32 are controls and are not shown.

The editor writes a line break as CRLF: the escape `#K(13)` followed by the newline code (byte `0x80`). In a raw dump, the escape `#K(13)` appears as the bytes `v*eUWf`, followed by a byte with the high bit set. That is why it looked like a "space token". It is a line break.

Correction

The old notes claimed two alphabets, a `char + 89` file rule with "subtract 6 from A–Z", and `v*eUWf` + high byte = space. All three were wrong. There is one alphabet indexed by `byte - 32`. `+ 89` only looked right for some characters. `v*eUWf` + `0x80` is `#K(13)` + newline. Viewers that drop non-ASCII bytes produce the garbled appearance seen on many house-outside signs. NPC scripts in the same file are plain Latin-1 with `0xA7` as the newline, not sign-encoded.

Drop-in decoder

`formats/sign-decoder/signText.js` (no dependencies; tests in `test.js` and `vectors.json`, from 11 real signs in GR-V1.03 levels):

- `decodeSignRecord(line)`: takes a raw sign line or a `PLO_LEVELSIGN` payload and returns `{ x, y, text }`.
- `decodeSignText(payload)`: takes the payload after the coordinates. By default the CR from `#K(13)` is dropped.
- `encodeSignText` / `encodeSignRecord`: the reverse.

It is not for GLEVNW01 text signs and not for NPC scripts.

GLEVNW01 signs

Text signs in a `.nw` file are plain. The SIGN record is `SIGN x y / text / SIGNEND`. The body may be empty. No alphabet transform is required.

Offline implication

An offline session that already parses SIGN records can display text correctly by:

- Leaving GLEVNW01 signs as plain text.
- Decoding binary signs from `.graal` or `.zelda` files with the alphabet above.

The same decoder covers `PLO_LEVELSIGN` if online mode is added later.

`formats/tile-walk.md`

Tile walk and collision

Date: 2026-10-14

Derived from observed client behavior, public tiletype tables, and format documentation. No decompile material.

Sheet loading

The client reads a tile definition list. Each entry is `image,prefix,type` with optional `x,y` offsets. The default sheet is `pics1.png` of type 0. Type 0 is the primary walk surface used by most classic levels. Type 1 and type 2 sheets use a separate rectangular grid for collision.

A level may reference multiple sheets. The active sheet is selected when the level is entered. Offline sessions that already load a GLEVNW01 board can select the same default sheet.

Collision rules

Observed type meanings:

- Type ≥ 20 blocks movement (wall).
- Type == 11 does not block movement (water). It changes the displayed tile and may set a swimming flag.
- Type 0 tiles are walkable only when their 12-bit id appears in one of two internal allow lists (commonly called the “nowall” lists). An id absent from both lists is treated as blocking even if the sheet is type 0.
- Type 1 and type 2 tiles consult a fixed grid. One extraction of that grid counted 1,725 blocking cells and 2,371 walkable cells.
- Throw skip is type 0x14. Objects whose shape type is above 19 block.

These rules match the offline session’s existing behavior of rejecting moves onto wall tiles and tracking water without blocking.

Type-0 allow lists

The two allow lists are populated when the sheet is loaded. They contain the tile ids that the original client treats as non-blocking on a type-0 sheet. The lists are not stored in `tiledefs.txt`; that file only names the sheet, prefix, and type. An offline session that wants exact classic walk behavior must either embed the same allow lists or derive them from a known-good `pics1` extraction.

One exported type-0 mask counted 3,195 non-blocking tiles and 682 blocking tiles. The type 1/2 grid counts above are independent of that mask.

Stamp vocabulary

Stamp id 0 is a 4×4 ground fill. Stamp id 1 is a hole and is never written. The 2×2 Bush stamp writes the four ids 2, 3, 18, 19. These ids appear in the type-0 walk tables and are therefore walkable when the sheet is type 0. A generator or editor that places stamps by name can emit the same ids without inventing tile codes.

Offline next step

The offline session already rejects wall tiles and tracks water. The remaining collision work is:

1. Select the sheet the original client would have selected (default `pics1.png` type 0).
2. Apply the type-0 allow lists when the sheet is type 0.
3. Fall back to the type 1/2 grid for other sheets.

Once that is done, walking an offline level exercises the same collision path the online client uses.

`formats/glevnw01.txt`

NAME

glevnw01 - Graal text level format (.nw)

DESCRIPTION

A *.nw* level is line-oriented text. The first line is the signature **GLEVNW01**. Official notes (*newfeatures2001.txt*) say to edit it in an editor that does not wrap lines. Servers send the same size payload as a binary *.graal* file; only the on-disk form is larger.

TLevel::loadLevel picks the loader from the extension (**.nw**,**.graal**,**.zelda**) or, if the extension is unknown, from the 8-byte magic:

GLEVNW01	text loader
GR-V1.01, GR-V1.02, GR-V1.03	binary Graal loader
Z3-V1.03, Z3-V1.04	Zelda loader

BOARD

```
BOARD x y width layerindex tilesdata
```

One horizontal run on the background. *tilesdata* is *width*2* characters. Each tile is a 12-bit index into the tile list (**pics1.png**), packed as two base64 characters, high 6 bits first.

```
alphabet = ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/
tile     = index(left) << 6 | index(right)
```

AA is tile 0, AB is tile 1, // is tile 4095. The client alphabet in *TLevel.cpp* appends a trailing =, which is not part of the official 64-character alphabet.

The loader ignores a BOARD line unless it has 6 tokens, the run stays inside x 0..63, and *tilesdata* is at least *width*2* characters. The tile is stored at *levelTiles[x + y*64]*. *layerindex* is parsed and not used.

Other records

LINK dest x y width height newx newy	level change rectangle
SIGN x y / text / SIGNEND	sign; body may be empty
NPC image x y / script / NPCEND	'-' if the npc has no image
BADDY x y type / verses / BADDYEND	type name or 0..9
CHEST x y item signindex	item name or 0..24

LINK destination, newx, and newy must not contain spaces. NPC x and y may be floating point. Baddy verses may be shortened, but BADDYEND is required. CHEST item names are resolved with **TLevelItem::getItemId**; an unknown item drops the chest.

Editor text

graaleditor(7) opens *.*graal* and *.*nw* with one filter and treats both as text. It does not require the **GLEVNW01** signature. **BOARD 0** is the tile layer: 64 lines of 64 tile indices, not a base64 run. Heights are a **TLevelHeights** overlay, not a second **BOARD**. Section keywords are the same words (**LINK**,**SIGN**,**CHEST**,**NPC**,**BADDY**,**LEVELNAMES**), but the field order is the order the dialogs collect. It is not a byte-for-byte dump of the editor's save function, and it is not what **TLevel::loadLevel** accepts. A file of raw indices will not pass the SDL loader until it is rewritten as **GLEVNW01** base64 runs. The other direction is the same mistake: a client *.nw* is not a **BOARD 0** grid of decimal indices.

Generator levels

terraingenerator(7) writes **GLEVNW01** with 64 rows of tile 0. The template at **0x44bbcc** is *64 <y>* and 128 **A** characters. The terrain is a **HEIGHTS** block, nine lines of nine numbers, closed by **HEIGHTSEND**, then a **LINK** on each edge that has a neighbor. There is no **NPC** section in that image.

dungeongenerator(7) writes **GLEVNW01** with one base64 **BOARD** of 64 by 64 shorts, edge **LINK** lines, and **NPC** blocks. Stamp id 0 is replaced by a 4 by 4 ground fill before the board is stored. Stamp id 1 is a hole and is not written.

Three writers, two of them sharing the signature. The editor's decimal **BOARD 0** is the third shape.

SEE ALSO

tiletypes(5), **graaleditor**(7), **terraingenerator**(7), **dungeongenerator**(7), **grmap001**(5), **2dadventureclient**(7), **graal**(7)

LINUXMAN 2

Carried from edition 1.1 and retitled. The map of this edition is **linuxman**(7). The book is *architecture.html*. New pages: **graalshop**(7), **gani**(5), **graallevelgen**(7), **rcprotocol**(7), **thinstall**(5).

`formats/gani.txt`

NAME

gani - GANI0001 sprite mode and GANI0002 movie mode

DESCRIPTION

A GANI file is the text document **graalshop**(7) reads and writes. The first line is the discriminator. Client samples under *levels/ganis/* and *levels/bomys/* match the sprite dialect, including files that omit the signature. The parser lowercases each line before comparing keywords. The writer emits uppercase keywords and space-padded columns.

First line	TGraalAni+0x20	Body
GANI0001	0	sprite table, header, ANI ... ANIEND
GANI0002	1	sprite table, header, MOVIE ... MOVIEEND

GRAMMAR

```
file        := signature? sprite* header* body
signature   := "GANI0001" | "GANI0002"
sprite      := "SPRITE" id sheet x y w h description
             | "ATTACHSPRITE"  id parent dx dy
             | "ATTACHSPRITE2" id parent dx dy
header      := "LOOP"
             | "CONTINUOUS"
             | "SINGLEDIRECTION"
             | "SETBACKTO" name
             | "BACKCOLOR" "#" hex6
             | "FRAMES" count
```

```
             | "DEFAULTHEAD" file
             | "DEFAULTBODY" file
             | "DEFAULTPARAM" n value
             | "DEFAULTATTR"  n value
             | "COLOREFFECT"  id args
             | "ZOOMEFFECT"   id args
             | "ROTATEEFFECT" id args
             | "STRETCHXEFFECT" id args
             | "STRETCHYEFFECT" id args
             | "EFFECTMODE"   id args
body        := ani | movie
ani         := "ANI" step* "ANIEND"
movie       := "MOVIE" actor* "MOVIEEND"
actor       := "ACTOR" name  frame* "ACTOREND"
frame       := "FRAME" index  token ("," token)*
step        := placement ("," placement)*
             | "WAIT" n
             | "PLAYSOUND" name x y
placement   := spriteId x y
             | "PARAM" n x y
             | "ATTR"  n x y
script      := "SCRIPT" rawlines "SCRIPTEND"
```

SINGLEDIRECTION is written only in sprite mode. **FRAMES** is written only in movie mode. **SCRIPT** / **SCRIPTEND** is copied through as text. It is not executed by the editor.

SPRITE ROW

The sprite-row writer emits fixed-width columns: id padded to 4, sheet name padded to 14, then x, y, w, h padded to 4, then the description. That is why client files look like:

```
SPRITE    0         SPRITES    0    0   24   12 shadow
SPRITE  200            HEAD    0    0   32   32 head up
SPRITE   20           SWORD   62    0   28   28 sword
```

ATTACHSPRITE and **ATTACHSPRITE2** hang a sprite on a parent with a pixel offset. They are not a second sheet.

SPRITE MODE

ANI opens the step list. **ANIEND** closes it. A step is either a comma-separated placement row, a **WAIT** hold, or a **PLAYSOUND** cue. A placement is a sprite id plus x and y, or a **PARAM** or **ATTR** slot plus x and y.

A step holds four direction rows, or one row when **SINGLEDIRECTION** is set. The wait counter starts at 1. Each **WAIT** adds to it.

MOVIE MODE

MOVIE opens the actor list. **MOVIEEND** closes it. Each **ACTOR** has a name and a run of **FRAME** lines closed by **ACTOREND**. The kind is not a keyword in the frame line. It is the integer on **TGraalActor**: 0 CHAR, 1 SPRITE, 2 SOUND, 3 TEXT.

A **FRAME** line is an index and a comma-separated token list. Those tokens become the *key=value* strings on a **TGraalActorMod**.

HEADER FLAGS

LOOP and **CONTINUOUS** are playback flags. **SETBACKTO** names the GANI to return to. **BACKCOLOR** is a *#rrggbb* fill behind the preview. **DEFAULTHEAD** and **DEFAULTBODY** reload the head and body **TSurfaceOwner** objects. **DEFAULTPARAM** and **DEFAULTATTR** seed the 30-row attribute grid. Effect lines (*COLOREFFECT*,*ZOOMEFFECT*,*ROTATEEFFECT*, *STRETCHXEFFECT*,*STRETCHYEFFECT*,*EFFECTMODE*) are stored as id plus arguments and applied at blit time.

SEE ALSO

graalshop(7), **graaleditor**(7), **gif2mng**(7)

`formats/grmap001.txt`

NAME

grmap001 - Graal gmap text, short form and generator form

DESCRIPTION

A *.gmap* is line-oriented text. The first line is the signature **GRMAP001**. Two writers in this edition emit it, and they do not emit the same keys.

Short form

dungeongenerator(7) writes, from **0x406760**:

```
GRMAP001
WIDTH <levels across>
HEIGHT <levels down>
GENERATED <bottom-right level name>
```

There is no **LEVELNAMES** block. The client rebuilds the grid from **GENERATED**, which names the last level, not the number 1. Level names are *LEVELSTART* , a column letter, and a 1-based row. See **dungeongenerator**(7).

Generator form

terraingenerator(7) writes, from **0x44c5f5**:

```
GRMAP001
WIDTH <levels>
HEIGHT <levels>
GENERATED 1
GENSEED <seed>
GENBASE <base>
GENEVENBORDERS <0 or 1>
GENHEIGHT <height>
GENCHAOS <chaos>
LEVHEIGHT <per-level height>
LEVCHAOS <per-level chaos>
MAPIMG <basename>.png
```

```
<one level name per row>
HEIGHTMAP
<coarse or fine samples>
HEIGHTMAPEND
RANDOMSEEDS
<one seed per level, when present>
RANDOMSEEDSEND
```

GENERATED 1 here is a flag, not a filename. Level names are basename, column *A* then *AA*, and a zero-padded row, plus *.nw*. **MAPIMG** is the overview the client shows on the map list. The PNG the tool saves from the preview dialog is a separate file.

HEIGHTMAP is the field. A sibling *_heights.txt* is the same numbers without the gmap wrapper, and the *.txt* branch of the save dialog writes that file. Each level file then carries a 9 by 9 **HEIGHTS** block. See **glevnw01**(5).

GraalLevelGen

The picture generator writes **GRMAP001** with **WIDTH** and **HEIGHT**, then a **LEVELNAMES** list of quoted filenames separated by commas. That is a third shape. Do not assume **GENERATED** means the same thing in all three files.

SEE ALSO

terraingenerator(7), **dungeongenerator**(7), **glevnw01**(5), **graal**(7)

LINUXMAN 2

Carried from edition 1.1 and retitled. The map of this edition is **linuxman**(7). The book is *architecture.html*. New pages: **graalshop**(7), **gani**(5), **graallevelgen**(7), **rcprotocol**(7), **thinstall**(5).

`formats/tiletypes.txt`

NAME

tiletypes - which tile ids block, and which are water

SYNOPSIS

server/include/tiletypes.h
share/tiledefs.txt

DESCRIPTION

The client does not inspect the tileset image. A board stores a tile **id** (0..4095). Collision is a second table, indexed by that id.

tiletypes.h holds *static const unsigned char tiletypes[4096]* copied from the standard Graal tileset, plus *tiletypes_len* = 4096. *share/tiledefs.txt* is the same 4096 bytes written in decimal, row-major, 128 values per line (the width of **pics1.png**).

The only tests this client runs

From *TLevel.cpp :*

```
bool TLevel::isOnWall(double pX, double pY)
{
    if (pX < 0 || pY < 0 || pX > 63 || pY > 63) return true;
    return tiletypes[levelTiles[int(round(pY)) * 64 + int(round(pX))]] >= 20;
}

bool TLevel::isOnWater(double pX, double pY)
{
    return (tiletypes[levelTiles[(int)pY * 64 + (int)pX]] == 11);
}
```

So, for this client:

- **wall** if the type byte is **>= 20** (also if the foot is outside the 64x64 board)
- **water** if the type byte is **== 11**
- **walkable** otherwise, including types the official client treats as special

Outside the board, **isOnWall** is true. **isOnWater** does not range-check; a bad coordinate reads off *levelTiles*.

Counts in this table

value	count	meaning here
0	823	walkable
3	40	stored, unused (official: chair)
4	4	stored, unused (official: bed, upper)
5	4	stored, unused (official: bed, lower)
6	12	stored, unused (official: swamp)
7	8	stored, unused (official: lava swamp)
8	30	stored, unused (official: near water)
11	47	water
12	67	stored, unused (official: lava)
20	129	wall (official: throw-through fence)
21	18	wall (official: jumping stone)
22	2914	wall (official: blocking)

Wall ids: 129+18+2914 = 3061. Water ids: 47. Walkable ids: 988.

What the official list called them

newfeatures2001.txt documents **setshape2** tile types. That list is what the bytes were named for. This client's movement only keeps the two predicates above.

0	nonblocking
2	hurting underground
3	chair
4	bed, upper side

5	bed, lower side
6	swamp
7	lava swamp
8	near water
9	near lava
11	water
12	lava
20	throw-through (fences)
21	jumping stone
22	blocking tile

Types 2 and 9 are in the official list and do not appear in this 4096-byte table.

Not the Graal.exe body mask

A decompile note shipped as *attachments/tiledef.txt* describes Graal.exe v2.05. That file is a different grid: a 0/1 body-block mask for tileset types 1 and 2 (1725 blocked, 2371 walkable). Stock selection there is *pics1.png,,0* and type 0 walks only if the id is in two sorted lists that the decompile did not emit. **setshape2** in that client blocks the body when the shape-tile type is greater than 19, which matches **>= 20** here. Throw handling there does **not** stop on type 0x14 (20). This SDL client has no such split: 20, 21, and 22 are all walls.

Id to picture

On **pics1.png** (2048x512, palette, 128 by 32 tiles of 16 px):

```
col = id % 128
row = id / 128
pixel = (col * 16, row * 16)
```

The first data line of *tiledefs.txt* is ids 0..127. It starts *0,0,20,20,...* so the top-left two tiles walk and the next two are walls. The 2x2 Bush stamp in **tileobjects**(7) writes ids 2, 3, 18, and 19, which this table marks 20, 20, 22, 22. The stamp file itself has no collision column.

FILES

artifacts/2dadventureclient-master/server/include/tiletypes.h
artifacts/2dadventureclient-master/docs/tiledefs.txt
/LinuxMan/share/tiledefs.txt
artifacts/graal/pics1.png
artifacts/graal/levels/tiledefs/tiledefsOffline.txt

SEE ALSO

2dadventureclient(7), **graal**(7), **glevnw01**(5), **gobjset01**(5)

LINUXMAN 2

Carried from edition 1.1 and retitled. The map of this edition is **linuxman**(7). The book is *architecture.html*. New pages: **graalshop**(7), **gani**(5), **graallevelgen**(7), **rcprotocol**(7), **thinstall**(5).